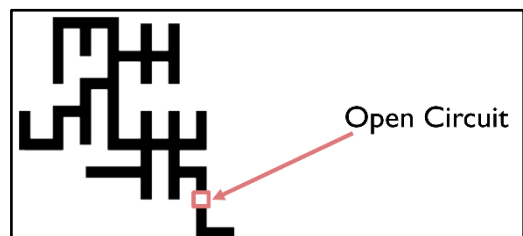

OPEN CIRCUIT DETECTION IN VLSI CIRCUITS USING DEEP LEARNING

Yuval Lifshitz (207639543) Tomer Morad (207132705) Dan Gur (207764937)

[GitHub & Dataset](#)

Introduction

Open circuit defects in VLSI chip fabrication are common and may prove fatal for a chip's functionality. Typically, they are identified during post-silicon testing using advanced (and highly expensive) imaging equipment, although methods which analyze chip photos and scans instead of interfacing directly with the



physical chip also exist, with varying effectiveness. With the success of deep learning in computer vision as well as its' rise in popularity, deep learning-based solutions are now being implemented and used for defect detection in VLSI as well. Our Project will use such methods to identify and locate open-circuit defects in synthetic images of VLSI circuits, which will provide a good basis for finetuning methods down the line. Our envisioned solution will receive high-quality scans and will be able to mark whether an open-circuit defect exists, and if it does exist, locate it.

Methodology

We present two approaches to defect detection, each solving a different computer vision task: classification and object detection.

Approach 1: Classification - CNN

- Model Training and Evaluation

We use a Convolutional Neural Network (CNN) to identify the coordinates of defects within circuit images. We define 48x48 distinct classes, each signifying a different pixel (coordinate) in the input image, as well as an additional "no defect" class. We train 4 versions of this model, each differing in number of layers, kernel size and size of the hidden dimensions. A breakdown of the size of each model can be found in Appendix A.

We integrate standard machine learning practices: convolution and pooling, batch normalization, etc. We use a cross-entropy loss, standard for classification tasks.

- Noise Injection and Robustness

We enhance the model's resilience by retraining it on a subset of the data injected with various types of noise which may arise when using physical imaging or scanning equipment.

We retrain each of the 4 model sizes on a mixed dataset in which we noised ~50% of the images. The exact methods of noising will be further expanded upon below.

Approach 2: Detection - YOLOv8

- Model Selection and Training

Our second approach shifted to a more modern object detection model: YOLOv8. YOLOv8 is an industry standard model for real-time object detection, making it efficient and lightweight, perfect for our use-case which utilizes very small images.

We used an existing implementation of YOLOv8 supplied by Ultralytics ([Ultralytics, 2025](#)), specifically “YOLOv8s”. YOLOv8 uses multiple adaptive losses (Focal, IoU, “Objectness”) to prevent overfitting ([Yaseen, 2024](#)). The YOLO model has a built-in classification stage in which it learns and decides whether a defect is present, only subsequently attempting to locate it in the image.

- Data Augmentation for Generalization

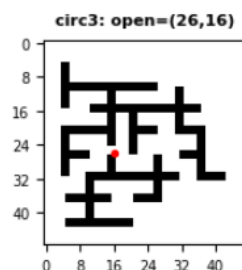
As a part of the training suite, the implementation provided by Ultralytics offers several advanced data augmentation techniques. As in the first approach, this allows the model to better generalize to real-world conditions with varying image quality.

Datasets

Developing a model for VLSI defect detection is challenging due to the lack of publicly available datasets. VLSI design data is proprietary and sensitive, which makes it nearly impossible to obtain. We address this issue by using a large, synthetic dataset.

The synthetic circuits are generated using algorithms which solve the “Minimum Rectilinear Steiner Tree” problem, expanded upon in Appendix B. This approach mimics the intricate routing patterns of real circuits, ensuring our synthetic data has a similar structure to real VLSI layouts.

We use a dataset comprising 500,000 synthetic circuit images, each rendered as a 48×48 binary image with one open defect. The dataset is divided into five HDF5 files, each holding 100,000 images, totaling 500,000 defective circuit representations. An example labeled image is provided, where the red dot indicates the location of the open defect.



Adding Data with No Defects

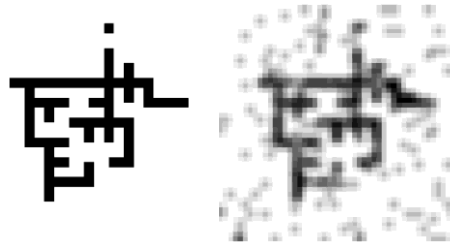
A model trained exclusively on defective circuits will incorrectly identify non-defective images as faulty. To provide our neural networks with accurate data that mirrors real-world scenarios, we include defect-free examples in our training set. We sourced an additional dataset of 10,000 images of rectilinear trees with no defects. We added these to our existing set of 500,000 defective images, such that we have 510,000 images in total. This expanded dataset ensures the model can learn to accurately distinguish between functional circuits and those with open defects, as intended.

Injecting Noise and Applying Modulations

Our synthetic data poses a challenge: real world images are not as perfect as the ones we have obtained. To address this, we inject noise and apply modulations on the data.

Noising for Approach 1:

As the size of the input layer and number of classes in this approach are fixed, we were limited in the forms of noising and modulations we could apply. For example, had we applied a rotation of $\theta \notin [90, 180, 270]$ degrees we could inadvertently create another open circuit as a result of non-overlapping pixel coordinates and interpolation, which would be unlabeled and thus confuse the model. We settled on three types of noise: Gaussian noise, Salt and Pepper and Gaussian Blur. We also considered adding 90° rotations and mirrorings, however, when scanning the dataset, we noticed that in many cases applying these modulations recreated a circuit that already existed, making it practically redundant. An example noised image can be seen below.



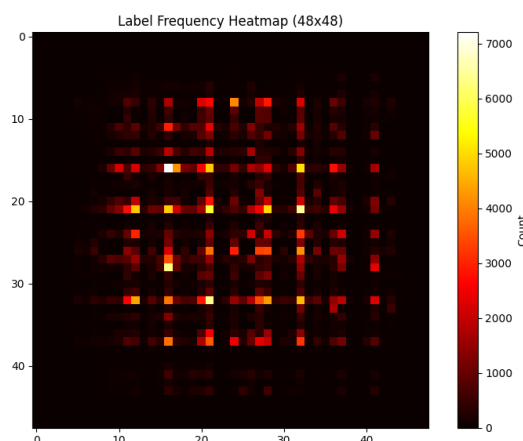
Noising and Modulations for Approach 2:

As previously stated, the provided implementation of YOLOv8 includes a suite of image modulations, such as distortions, rotations, brightness changes and image wrapping, seen below.



Class Distribution

Through a simple analysis of the dataset, we notice that the classes are mostly distributed around the center, with very few cases of the defect appearing along the edges of the image. This is exemplified in the following heatmap and will be relevant when evaluating the model later.



Bounding Box Rescaling for Approach 2

The dataset provides defect annotations as pixel coordinates in 48×48 image space, while YOLOv8s expects inputs with dimensions 640×640. We implement a normalization pipeline to convert these to YOLO's required format. Given bounding box coordinates (x_1, x_2, y_1, y_2) , we computed normalized center coordinates as $\left(\frac{x_1+x_2}{2 \cdot 48}, \frac{y_1+y_2}{2 \cdot 48}\right)$, with normalized dimensions $\frac{x_2-x_1}{48}$ and $\frac{y_2-y_1}{48}$. This transformation ensures scale invariance when images are resized to 640×640 for training, maintaining spatial relationships between defect locations and image boundaries.

Training and Evaluation

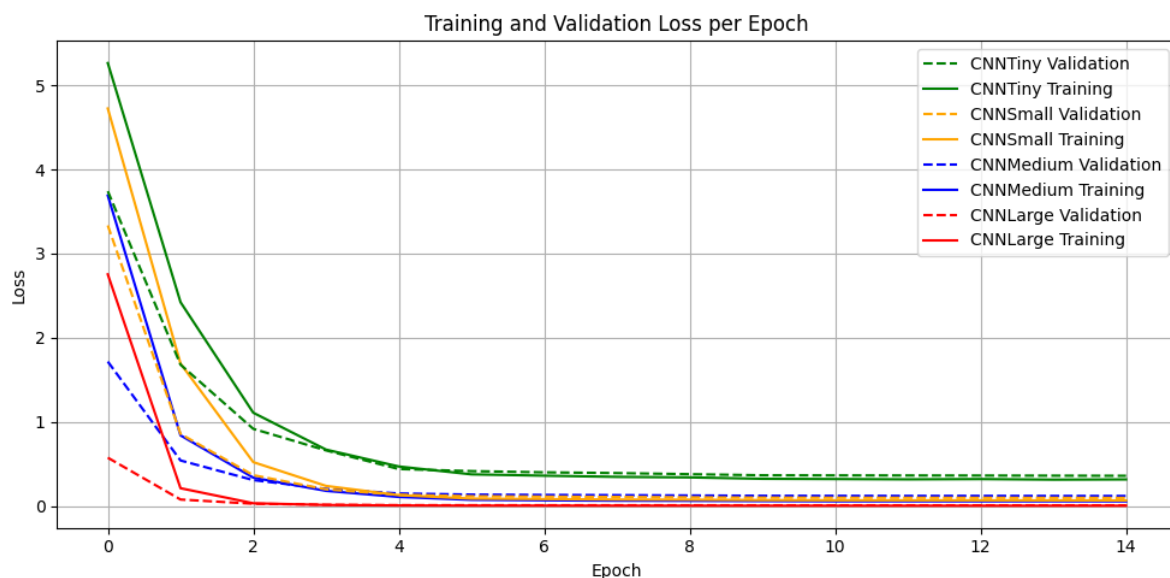
For approach 1, we train the model using the school's Slurm cluster with any available GPU (RTX 2080 Ti, RTX 3090, Tesla V100). The model is trained for 15 epochs with a train-val-test split of 80%-10%-10%.

For approach 2, we train the model using Google Colab with a Tesla A100. The model is trained for 20 epochs on a subset of our dataset and using a train-val-test split of 70%-20%-10%.

For the first approach we measure performance by achieved accuracy in classifying defects and the ability to pinpoint the exact coordinates (class) over the images in the test split of our dataset. For the second approach we measure precision and recall for the classification portion and IoU of our predicted bounding box compared to the label for the localization portion.

Results

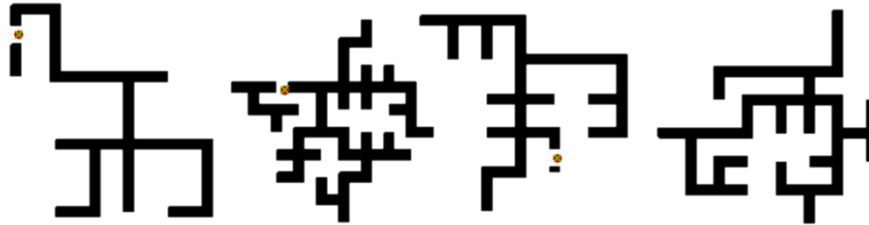
Approach 1 – Clean data



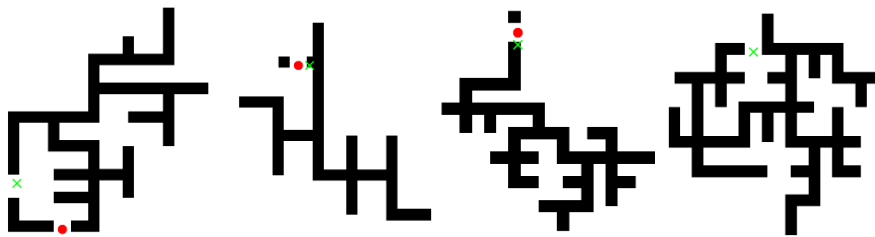
In the above convergence graph, as expected, the largest model converges the quickest. As we theorized, all versions converge to <1 loss and achieve >90% accuracy as seen in the table below. This indicates that the smaller versions may be adequate in solving this problem over the given dataset. Surprisingly, prior to convergence, the models achieve a better validation loss than training

loss, alluding to a similarity between the train and val splits. We present some samples below, where the red dot indicates the ground truth and the green “X” the predicted label.

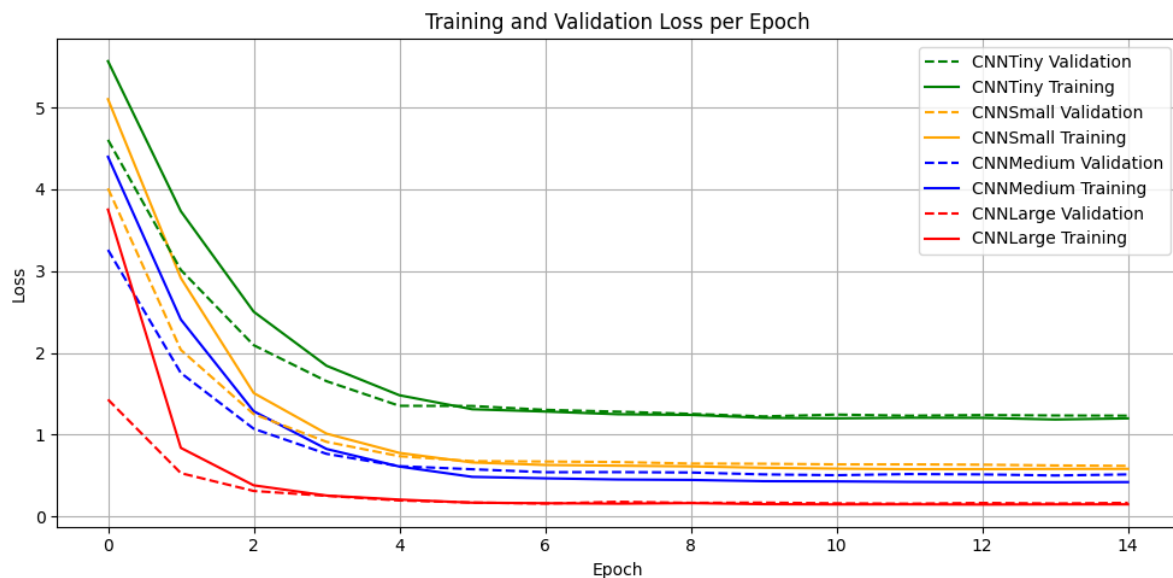
Model	CNNTiny	CNNSmall	CNNMedium	CNNLarge
Accuracy [%]	92.1	98.4	99.1	99.9



We challenge the models by sampling expected failure cases - indicated to us by the skewed class distribution presented previously - which we generate manually by shifting existing circuits, pushing the open circuit coordinate to the edges of the image or to spots with less frequent opens. As expected, the models fail to correctly classify these manually generated “out of distribution” samples, as can be seen below.

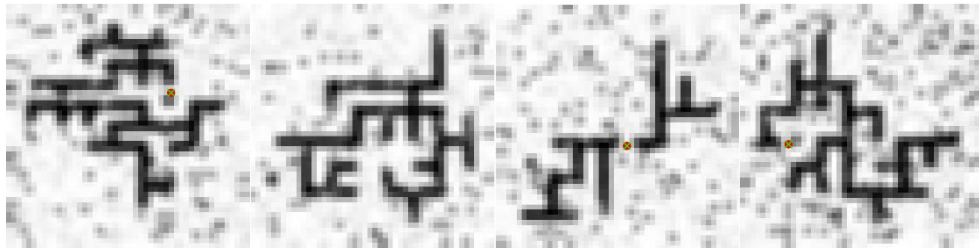


Approach 1 - Noised



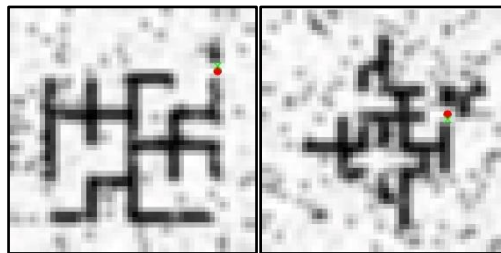
Similarly to the previous graph, all models converge quickly with the largest one converging the fastest. It is immediately apparent that incorporating noise altered the training process, as the distance between each of the plots widened. We also notice that each model converges to a higher cross-entropy and all except CNNLarge achieve a worse accuracy, shown in the table below. We also present some samples below.

Model	CNNTiny	CNNSmall	CNNMedium	CNNLarge
Accuracy [%]	80.1	95.4	98.9	99.9



When analyzing the failure cases which cause the degradation of accuracy, we noticed the model frequently predicts coordinates which are ± 1 pixel away from the correct label. Thus, when expanding our “search radius” and considering pixels in a 3x3 box around the label, we achieve results which are similar to the unnoised model. This indicates that although the noised model loses precise accuracy, for practical purposes (± 1 pixel) it is still viable, even with the smaller versions. Examples of such failure cases can be seen below.

Model	CNNTiny	CNNSmall	CNNMedium	CNNLarge
Accuracy [%]	91.7	98.3	99.0	99.9

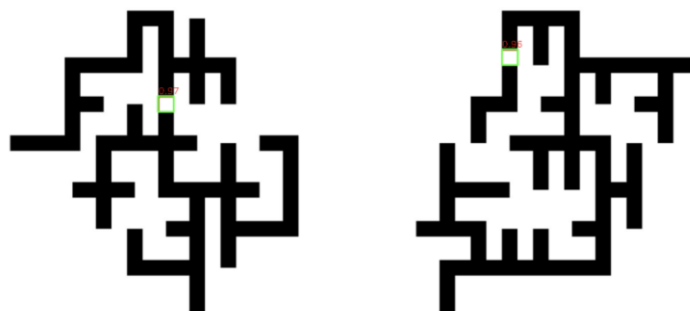


Approach 2 - Yolo

The YOLOv8 model was successful in both the classification and localization portions, achieving nearly 100% accuracy in classification and 98% mean IoU, as can be seen in the following table.

	Mean [%]	Std. Dev. [%]	Min. [%]	Max. [%]
Confidence	96.8	0.6	90.5	98.4
IoU	98.5	0.3	97.4	99.5

We believe this indicates that the task is very simple for the model. Another indication for this is the relative number of parameters: YOLOv8s has ~11M as compared to CNNLarge - which also achieves ~100% accuracy - with ~2.8M and a much simpler architecture and training pipeline. In the pictures below, we see the predicted bounding box (in red) nearly overlapping the ground truth bounding box (in green), as expected from the table. This supplies further, visual proof for our hypothesis



Conclusions and Future Work

The models we present in this report, as they stand, cannot be applied to real scans of VLSI circuits. Real scans, visualized in Appendix C, are much more complex, expressive, with larger ranges of contrast, multiple channels and higher resolutions. So long as a compiled, open-source and fair-use dataset does not exist there will not exist an open-source model which solves this task and can be used practically or commercially.

We do believe, however, that this project may serve as motivation for a simplified training pipeline when such a dataset becomes available: begin by pretraining your model on a synthetic dataset resembling the one we used (fitted for the correct resolutions/channels/etc.). Then proceed to a shorter fine-tuning stage using the available data. This may also allow training on a smaller dataset of real scans, which may be easier to compile.

Until this happens, there are a few ways to further improve our approaches. Firstly, in the current state of the dataset, we have circuits with either 1 or no defects. As circuits which contain multiple defects are expected in real scans, we should incorporate them into our synthetic dataset to further increase the generalization of our models to the real world. Secondly, to improve the classification approach, we should consider adding a “shifting” modulation to the noised model. This could solve the problem we showcased with the class distribution of the current dataset, further strengthening our model. Lastly, we believe it could be interesting to also train a “YOLOv8n”, an even smaller version of YOLOv8 with 3.2M parameters and seeing if it still performs as well.

Bibliography

Ultralytics. (2025). *YOLO GitHub Repository*. Retrieved from

<https://github.com/ultralytics/ultralytics?tab=readme-ov-file>

Yaseen, M. (2024). [What is YOLOv8: An In-Depth Exploration of the Internal Features of the Next-Generation Object Detector](#).

Appendix

A. CNN size comparison

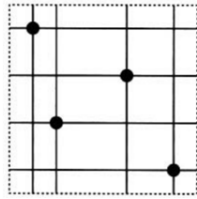
Class	# Layers (Conv. + FC)	# Parameters
CNNTiny	2 + 1	~670K
CNNSmall	3 + 1	~1.3M
CNNMedium	5 + 1	~2M
CNNLarge	5 + 1	~2.8M

B. Minimum Rectilinear Steiner Trees

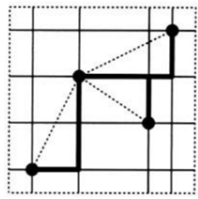
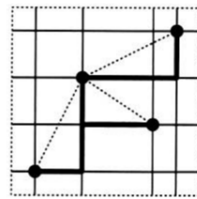
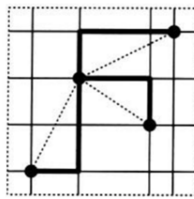
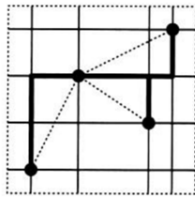
The Minimum Rectilinear Steiner Tree problem is a problem in geometry defined as follows: given n points on a grid, find a tree comprised only of vertical and horizontal line segments which connect the points while also having the least total segment length. It can be shown that each node in such a tree is either an input point or a “Steiner point”.

The problem arises in the physical design of electronic design automation: In VLSI circuits, wire routing is carried out by wires running only in vertical and horizontal directions, due to the high computational complexity of the task.

Below is an example of four possible solutions each constituting a Rectilinear Steiner Tree:



**Underlying grid
formation**



**Four examples of different Steiner Trees constructed from a minimum
cost tree.**

C. Real VLSI Scan Visualization

